Part 2: Closing the Loop

This section wasn't part of the course when I first took it, so I refer to it as *Closing the Loop*. It is also because it marks the final work to complete the awesome Deep Learning series. Thanks to Yue for giving me the opportunity to complete this part at a later time.

1. Data Generation:

- Similar to Part 1, we write code to generate data that follows the format of the two sample files, one for training and one for validation.
- The data generation functions from Part 1 can be reused with slight modifications. Unlike Part 1, we don't generate QA pairs; instead, we focus on generating potential captions for images.
- I ended up generating around 210k captions.

2. The Oversight:

- In the lectures, we learned about the ***[CLS]*** token, which is located at the beginning of the input text and is often used to represent the entire sequence for classification tasks.
- This is also standard practice in many large language models. If you use GenAI tools to assist with code, they often default to using the CLS token as the text representation.
- However, this model doesn't follow that convention. You'll need to investigate further, and average pooling might be a simple and effective alternative.
- The loss and forward function largely follow the approach outlined in this paper: https://arxiv.org/pdf/2103.00020 (page 5 - screenshot below)

3. Trick #1 – The Path to Full Marks:

- Through the lectures and earlier assignments, we have learned that the attention mask is crucial, especially for transformer-based text encoders.
- Review the text_encode function provided and see if you can make improvements based on this understanding.

4. GPU, Batch size, LoRa:

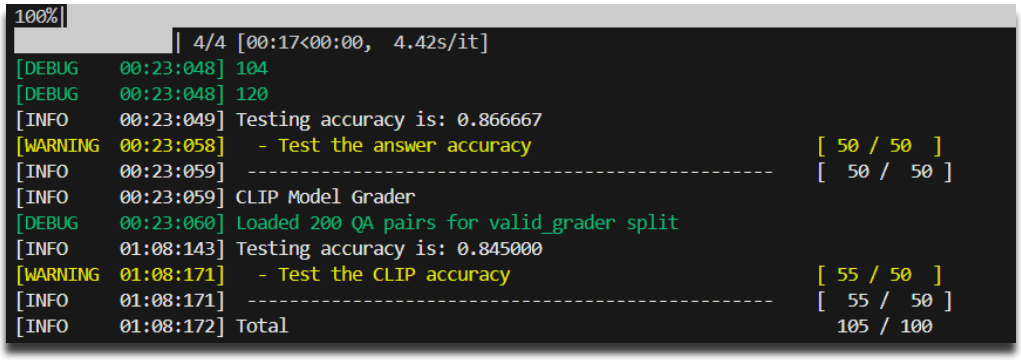
- I can get extra credits with the default batch size using L4 on Lightning.ai or the significantly smaller batch size with accumulative gradient using my laptop GPU. I think it can work well for a humble 6GB GPU.
- You can also play around with parameters like LoRa and other hyperparameters like previous assignments.

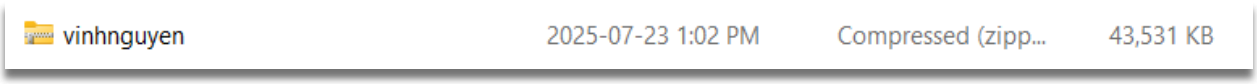
5. Trick #2 – Ending on a High note:

- There are various ways to earn extra credit, and here is one that worked, at least for me.
- Take a close look at **clip.py** and consider whether the tokenization is implemented in the best way. Identifying and fixing a poor practice here could boost your model's accuracy by 5–10%. I got 72-73% without fixing it and 85% after fixing it.

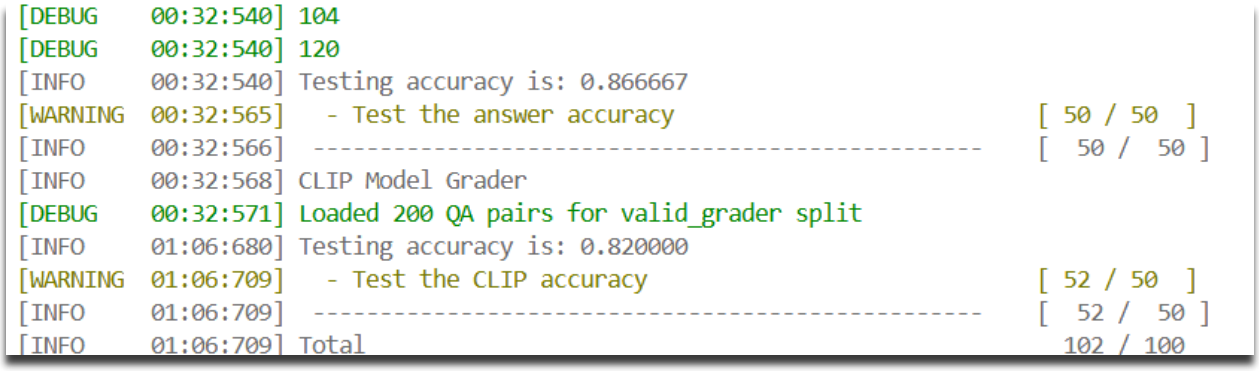
6. Result:

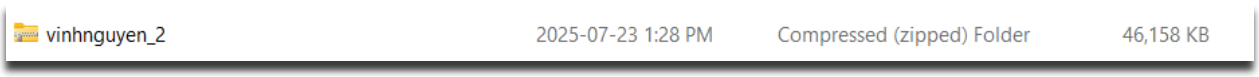
- Option 1: batch_size = 256, gradient_accumulation_steps = 4 (suitable for 6GB GPU)





- Option 2: default batch_size = 1024, using simple average pooling representation.





```
# image_encoder - ResNet or Vision Transformer
# text_encoder  - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l]       - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t             - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss   = (loss_i + loss_t)/2
```